Listas

Estadística con Python

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

1. Introducción
2. Creación de listas
3. Indexado
4. Edición
5. Métodos asociados a listas
6. Sub-listas
7. Comprensión de listas
8. Copiar listas vs. asignar referencias (`Aliasing`)
9. Ejemplo de aplicación
10. Listas anidadas



Introducción



Introducción



Introducción

Las listas sirven para agrupar objetos de diferente tipo, aunque típicamente se usan para alojar datos de la misma naturaleza (números, caracteres, lógicos, ...), son indexadas y, a diferencia de las tuplas, son **mutables**, es decir, se les puede agregar, quitar y cambiar elementos después de creadas. Las listas soportan todas las características de las tuplas, pero es posible adicionar, remover y modificar sus elementos.



Creación de listas



Creación de listas

Para crear una lista colocamos los elementos dentro de corchetes de la siguiente forma:

```
l1=[-5,-4,-3,-2,-1,1,2,3,4,5] # lista de números enteros
l2=[2,"a",True]    #lista con elementos de distinto tipo
print(l2,type(l2))
```

```
[2, 'a', True] <class 'list'>
```



Creación de listas (cont)

También se pueden crear listas a partir de otros objetos previamente creados, por ejemplo tuplas, con la función `list()`

```
t=(2,3,4,5,6)
list(t)
```

```
[2, 3, 4, 5, 6]
```

```
list(range(10))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



Indexado



Indexado de listas

- Cada elemento de una lista está asociado con un *índice* que refleja su posición en el arreglo.
- Como en las tuplas, el primer elemento tiene índice 0, el segundo índice 1 y así sucesivamente.
- Asociados con la lista `l1` anterior tenemos 10 índices, iniciando en 0 y terminando en 9 .
- Para acceder al elemento con índice 3, es decir, al **cuarto** elemento de la lista, escribimos `l1[3]` que corresponde a -2



Indexado de listas, ejemplos

```
print(l1)
```

```
[-5, -4, -3, -2, -1, 1, 2, 3, 4, 5]
```

```
l1[3] # Cuarto elemento de la lista l1
```

```
-2
```



Indexado de listas (cont)

Análogo a las tuplas, estudiadas en la clase anterior, con las listas es posible usar **índices negativos**, lo cual conduce a indexar por la derecha. `l[-1]` corresponde al último elemento de la lista, `l[-2]` corresponde al anterior a `l[-1]` y así sucesivamente.

```
l1[-1] # Último elemento de la lista l1
l1[-2] # Penúltimo elemento de la lista l1
```



Edición



Inserción, adición y borrado de elementos en una lista

A diferencia de las tuplas, las listas se pueden modificar insertando, adicionando o borrando elementos, veamos como se usa el método `insert()` para insertar el número -6 como primer elemento de la lista `l1`

```
l1.insert(0,-6)
print(l1)
```

```
[-6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5]
```



Inserción (cont)

El primer argumento en la sentencia `l1.insert(0,-6)`, corresponde al índice donde se insertará el número dado en el segundo argumento (`-6`). La lista `l1` ahora tiene un elemento más, -6, en el índice 0 y el número -5, que estaba en este índice, pasó al índice 1.



Adición de elementos

Para agregar un elemento al final de la lista se usa el método `append()` como se muestra a continuación

```
l1.append(6)
print(l1)
```

```
[-6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6]
```

note que se ha agregado `6` al final de la lista `l1`.



Adición de elementos (cont)

Es posible adicionar más de un elemento al final de una lista concatenándola con otra lista, usando el operador +, análogo a como se hace con tuplas.

en este caso la lista `l2` no se modifica, para conseguirlo usamos

```
l2=l2+[4,5,6]
print(l2)
```

```
[2, 'a', True, 4, 5, 6]
```



Borrado

Para borrar un elemento de una lista se usa `del lista[i]` donde `i` corresponde al índice del elemento que se desea eliminar. A manera de ejemplo, borremos el elemento `'a'` de la lista `l2`, (`i=1`)

```
del l2[1]
print(l2)
```

```
[2, True, 4, 5, 6]
```



Borrado (cont)

Mientras que `del` elimina elementos de una lista por posición, el método `remove()` los elimina por valor. Para ilustrarlo, seguidamente borramos el elemento `4` de la lista `l2`.

```
l2.remove(4)
print(l2)
```

```
[2, True, 5, 6]
```



Borrado (cont)

Si en la lista hay más de un elemento con el argumento entregado a `remove()`, se elimina el de índice más pequeño, es decir, el primero que aparezca en la lista, veamos el siguiente ejemplo

```
l3=[2,5,4,-1,4,3,0,-2]          ①
l3.remove(4)                    ②
l3
```

① El número 4 está en la lista dos veces
② Se elimina el primer 4

```
[2, 5, -1, 4, 3, 0, -2]
```



Borrado (cont)

Otro método para eliminar elementos de listas es `pop()` que recibe un índice, elimina el elemento que corresponde, pero además regresa el valor eliminado

```
l3.pop(2) # regresa -1 (índice 2)
```

```
-1
```

```
print(l3) # l3 queda modificada, sin -1
```

```
[2, 5, 4, 3, 0, -2]
```



Borrado (cont)

Si no se entrega índice, el método `pop()` remueve (y regresa) el último elemento de la lista.

```
l3.pop()  # regresa -2 (el último elemento de l3)
```

```
-2
```

```
print(l3) # l3 queda modificada, se remueve -2
```

```
[2, 5, 4, 3, 0]
```



Longitud de una lista.

Cuando queremos recuperar en número de elementos que contiene una lista, procedemos de manera similar a como se hace con tuplas, usamos la función `len()`

```
len(l2)
```

```
4
```

como una aplicación, podemos combinar las funciones `del` y `len()` para borrar el último elemento de una lista

```
del l2[len(l2)-1]
```



Métodos asociados a listas



Indexación de listas

En esta sección se estudian las distintas formas de acceder, recuperar y cambiar elementos de una lista.



Verificar si un elemento pertenece a una lista

De manera análoga a como se hizo con las tuplas, se puede verificar si un elemento pertenece a una lista con el operador `in`. Si el elemento dado está en la lista se obtiene `True`, en caso contrario, `False`

```
print(0 in l1, 3 in l1)
```

```
False True
```



Índice de un elemento.

Con el método `index()` se recupera el índice que corresponde a un valor en la lista. Si el valor está en la lista más de una vez, se regresa el primer índice.

```
print(l1)
```

```
[-6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6]
```

```
l1.index(-1) # -1 tiene índice 5
```

```
5
```

```
l3=[2,5,4,-1,4,3,0,-2] # en esta lista 4 está dos veces
l3.index(4) # índice del primer 4
```

```
2
```



Indexación de listas (cont)

- En caso que el argumento entregado al método `index()` no esté en la lista, ocurre un error.
- Por ejemplo, `l3.index(0.5)` regresa `ValueError: 0.5 is not in list`, esto es lo que se conoce técnicamente como una excepción no controlada (uncaught exception).
- Para evitar que tu código se `reviente`, podrías indagar si el elemento está en la lista, en caso afirmativo usas `index()`, en caso negativo, imprimes un mensaje



Indexación de listas (cont)

```
i=0.5
if i in l1:
  pos=l1.index(i)
  print(f"{i} se encuentra en la posición {pos}")
else:
    print(f"{i} no está en la lista")
```



Capturar errores con `try-except`

Tip

Como se La forma **técnica** de tratar con este error en python es capturándolo con `try-except`:

```
i=0.5
try:
    pos = l1.index(i)
    print(f"{i} se encuentra en la posición {pos}")
except ValueError:
    print(f"{i} no está en la lista")
```

Con `try`, python intenta ejecutar el código dentro del bloque. Si ocurre un `ValueError`, python salta al bloque `except` y ejecuta ese código.



Sub-listas



Sublistas

Para extraer una sublista de una lista empleamos la sintaxis `l[i:j]`, `i` corresponde al índice del elemento inicial a partir del cual, incluido, comenzará la sublista, mientras que `j` indica el extremo superior, la sublista llegará hasta el elemento con índice `j-1`. La sentencia `l[i:]` regresa una sublista de `l` que inicia en el índice `i` hasta el final de la lista, de manera análoga `l[:j]` es una sublista que inicia en el índice cero de la lista hasta el índice `j-1`, por último `l[:]` regresa la lista completa.



Sublistas

Veamos

```
l1
```

```
[-6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6]
```

```
l1[2:7]   ①
l1[2:]    ②
l1[:4]    ③
l1[:]     ④
```

① Desde el elemento con índice 2 hasta el elemento con indice 6.
② Desde el elemento con índice 2 hasta el último.
③ Desde el primer (índice 0) hasta el elemento con índice 3.
④ La lista completa.



Otras formas de crear listas

Es posible construir listas a partir de `tuplas`, `diccionarios`, `sets` (conjuntos) y otros objetos, mediante la función `list()`. Veamos un ejemplo de creación de listas a partir de tuplas

```
t1=(3,4)
lt1=list(t1) # se crea lista a partir de la tupla t1
print(type(lt1))
```

```
<class 'list'>
```



Combinación de las funciones `list()` y `range()`.

En el siguiente ejemplo se usa la función `range()` que representa una secuencia *inmutable* (como las tuplas), estrictamente creciente o decreciente, de números. Esta función se utiliza en bucles `for` para realizar un número específico de iteraciones en función de un valor que se reduce o aumenta en una constante (`step`), hasta que se alcanza un límite. La función `range()` regresa un objeto de tipo `range`, pero se puede convertir en una lista mediante la función `list()`.



Combinación de las funciones `list()` y `range()`.

Ilustremos su uso con ejemplos

```
print(range(9),type(range(9)))   ①
list(range(9))                   ②
list(range(3,22,2))              ③
```

① La clase del objeto `range(0, 9)` es `'range'`
② Lista que contiene secuencia de enteros desde cero (0) hasta ocho (8).
③ Lista que contiene secuencia de enteros desde 3 hasta 21 incrementando en 2.



Combinación de las funciones `list()` y `range()`.

- En `range(start,stop,step)`, `start` es el parámetro inicio, entero a partir del cual se devolverá la secuencia,
- `stop` es el entero que indica el final; la secuencia termina en el mayor entero que es menor o igual que `stop-1`,
- Por último `step` es el paso, que determina el incremento entre cada entero de la secuencia.
- Se puede iterar sobre el objeto (`range`) sin necesidad de convertirlo a una lista, uso que es bastante frecuente cuando se programa en python



Combinación de las funciones `list()` y `range()`.

```
for i in range(3,15,3):
  print(i)
```



Comprensión de listas



Comprensión de listas

La comprensión de listas es una forma elegante de definir y crear listas basadas en listas existentes. Es una forma concisa de generar una lista al aplicar una operación a los elementos de otra lista, esto se ilustra en el siguiente ejemplo:

```
[-2+0.5*x for x in range(9)]
```

```
[-2.0, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2.0]
```

Lo que se hace en el código anterior es evaluar la expresión $-2 + 0.5x$ para cada valor x en la secuencia generada por la función `range()`.



Comprensión de listas (cont)

Otro ejemplo, suponga que tiene una lista con valores de temperatura expresada en grados centígrado (C) y se le pide que haga la conversión a grados Fahrenheit (F) mediante la transformación $F = \frac{9}{5}C + 32$, podemos hacer la conversión de la siguiente forma

```
C=[25,32,36,19,20,24] #temperaturas en grados centígrados
F=[9.0/5*x+32 for x in C] #temperaturas en grados Fahrenheit
F
```

```
[77.0, 89.6, 96.8, 66.2, 68.0, 75.2]
```



Comprensión de listas (cont)

En la comprensión de listas se pueden utilizar sentencias condicionales para generar nuevas listas, veamos un ejemplo: de la lista l3 obtener una sub-lista que contenga sólo los elementos positivos.

```
print(l3)
```

```
[2, 5, 4, -1, 4, 3, 0, -2]
```

```
[x for x in l3 if x>0]
```

```
[2, 5, 4, 4, 3]
```



Copiar listas vs. asignar referencias (`Aliasing`)



listaB=listaA

Tenemos la lista l3

```
[2, 5, 4, -1, 4, 3, 0, -2]
```

Igualamos l4 a l3

```
l4=l3
```

Modificamos l4 agregando 6 al final

```
l4.append(6)
```

Imprimimos l3

```
print(l3)
```

```
[2, 5, 4, -1, 4, 3, 0, -2, 6]
```



listaB=listaA.copy()

Tenemos la lista l3

```
[2, 5, 4, -1, 4, 3, 0, -2, 6]
```

Hacemos l4=l3.copy()

```
l4=l3.copy()
```

Modificamos l4 agregando 7 al final

```
l4.append(7)
```

Imprimimos l3

```
print(l3)
```

```
[2, 5, 4, -1, 4, 3, 0, -2, 6]
```



Importante

Tip

- Si se hace `lista2 = lista1`, lo que se conoce como `creación de alias (Aliasing)` o `asignación por referencia`, al modificar `lista2` también alterará `lista1`.
- El método `.copy()` (ej. `lista2 = lista1.copy()`) crea `lista2` como un objeto independiente. Esto se conoce como `copia superficial` o `Shallow Copy`



Ejemplo de aplicación



Aplicación en control de calidad

Supongamos que monitoreamos el grosor de piezas en un proceso de manufactura. Almacenamos las mediciones de las muestras en una lista.

```
# 1. Recolección de datos
mediciones = [10.2, 10.1, 9.8, 10.3, 11.5, 9.9, 10.0, 10.1, 8.5, 10.2]

# 2. Uso de funciones nativas para calcular la Media
n = len(mediciones)
suma_total = sum(mediciones)
media_muestral = suma_total / n

print(f"La media del proceso es: {media_muestral:.3f}")
```

```
La media del proceso es: 10.060
```



Identificando anomalías

Si una medición está por encima de $\overline{x} + 0.5$ o por debajo de $\overline{x} - 0.5$ se considera una anomalía y debemos identificarlas

```
# Establecemos límites de control empíricos
limite_sup = media_muestral + 0.5
limite_inf = media_muestral - 0.5

# Extraemos solo las piezas que están fuera de control
piezas_defectuosas = [x for x in mediciones if x > limite_sup or x < limit

print(f"Mediciones fuera de control encontradas: {piezas_defectuosas}")
```

```
Mediciones fuera de control encontradas: [11.5, 8.5]
```

```
print(f"Total de defectos: {len(piezas_defectuosas)}")
```

```
Total de defectos: 2
```



Listas anidadas



Listas anidadas

Como con las tuplas, es posible tener dentro de una lista otras listas o tuplas u objetos de distinto tipo, a continuación se crea una lista que *anida* las listas `C` y `F` creadas anteriormente

```
T=[C,F]
```



Listas anidadas (cont)

Para recuperar el tercer elemento de la lista `F`, que está anidada en `T` se usa la indexación, recuperando primero a `F` y luego al elemento dentro de `F`

```
T[1][2]
```

```
96.8
```



Ordenar una lista.

El método `sort` ordena los elementos de una lista, por defecto lo hace de manera ascendente, es decir, de menor a mayor, si se quiere que el orden sea descendente usamos la opción `reverse=True`.

```
l3
```

```
[2, 5, 4, -1, 4, 3, 0, -2, 6]
```

```
l3.sort()
l3.sort(reverse=True) #orden descendente
```

